phone_app/lib/phone_app/conversations/query/sms_message_store.ex
```
def create_sms_message(params) do
  {:ok, contact} = ContactStore.upsert_contact(params)

  params
  |> Map.merge(%{contact_id: contact.id})
  |> SmsMessage.changeset()
  |> Repo.insert()
end
```

The first line retrieves the contact from the database so that we can pass its id field into our changeset.

This pattern for a create function is very useful: start with the provided parameters, overwrite (merge) any fields that the caller doesn't need to provide, turn it into the appropriate changeset, and then call Repo.insert/1 with that changeset.

When you use this pattern, you get the opportunity to overwrite the values of the params argument. In our case, we find or create the associated contact, so we fill out the contact_id in the changeset parameters.

Next, let's write a function that updates an existing SMS message. This is necessary because we'll track the external status of the message as it flows through the SMS network. Add the following code:

phone_app/lib/phone_app/conversations/query/sms_message_store.ex
```
def update_sms_message(message_sid, update_params) do
  case Repo.get_by(SmsMessage, message_sid: message_sid) do
    nil ->
      {:error, :not_found}

    existing ->
      update_params
      |> SmsMessage.update_changeset(existing)
      |> Repo.update()
  end
end
```

This function starts by calling Repo.get_by/2. This function looks up a schema by fields other than its primary key. In our case, we look up by a single field (message_sid), but we could look up by multiple fields.

We need to look up the associated schema so that we can call update_changeset/2 with it. Ecto is smart enough to know which fields have changed, and it sends only changed fields to PostgreSQL.